



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECP3843

Special Topic in Data Engineering

Tutorial 2: Apache Spark

LECTURER: DR. ARYATI BINTI BAKRI

NO.	NAME	MATRIC NUMBER
1.	IMAN ABADI BINN MOHD NIZWAN	A23CS0084
2.	MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112

Table of Contents

1. Business Understanding.....	3
2. Data Understanding.....	3
3. Data Preparation.....	11
4. Modeling.....	14
5. Evaluation.....	17
6. Reflection.....	20

1. Business Understanding

1.1 Project Background

The Brazilian National Institute for Educational Studies and Research (INEP) conducts the National Basic Education Census every year. This census gathers detailed information on every basic education school across Brazil from many perspectives, including ethnic, geographical and economic dimensions. The dataset is made freely for everyone to use through the Brazilian Open Data Portal. But, the data is in raw form making it not usable for analysis.

1.2 Problem Statement

Educational researchers need an efficient way to explore and aggregate twelve years of Brazilian school census data. The raw CSV files are too big, too redundant and too slow to query directly. A scalable ETL (Extract, Transform, Load) pipeline is needed to convert these files into an analytical star schema that supports fast aggregate queries and dashboarding.

1.3 Business Objectives

To address the challenges of handling large-scale datasets, the project is guided by these business objectives:

- Build an ETL pipeline using that can ingest, clean and reshape twelve years of school census data.
- Deliver the transformed data as a star schema inside a database and ready for connection to BI (Business Intelligence) tool.
- Enable to answer analytics, aggregate business questions and visualize the questions accurately.

2. Data Understanding

2.1 Data Source

The data is downloaded from the Brazilian Open Data Portal by INEP (<https://www.gov.br/inep/pt-br/aceso-a-informacao/dados-abertos/microdados>). The portal publishes one ZIP file format archive per year of the education census. Each ZIP file contains a CSV file that lists every school in Brazil together with hundreds of attributes.

2.2 Data Characteristics

Table 1: Data characteristics of School Census Dataset

Attribute	Value
Source format	CSV
Time range	2010 – 2021
Columns per file	≈ 370
Size per file	≈ 180 MB
Total raw size	≈ 2.2 GB
Grain (one row)	One school in one census year
Approximate row count (all years)	≈ 3 million records after combined

2.3 Data Dictionary

The tables below show all the tables in the School Census dataset. It shows the variables, data type, length, nullable, constraint and description.

Table 2: Table state

Variable	Data Type	Length	Nullable	Constraint	Description
co_uf	INT	—	NO	PK	Unique numeric code for the state
no_uf	VARCHAR	50	NO	—	Full name of the state

sg_uf	CHAR	2	NO	—	Two-letter state abbreviation
--------------	------	---	----	---	-------------------------------

Table 3: Table municipality

Variable	Data Type	Length	Nullable	Constraint	Description
co_municipio	INT	—	NO	PK	Unique numeric code for the city
no_municipio	VARCHAR	100	NO	—	Full name of the city
co_uf	INT	—	NO	FK → state	Reference to the parent state

Table 4: Table school

Variable	Data Type	Length	Nullable	Constraint	Description
school_id	INT	—	NO	PK, AUTO_INCREMENT	Substitute key generated during ETL

co_municipio	INT	—	NO	FK → municipality	Reference to the city the school belongs to
tp_dependencia	TINYINT	—	NO	—	School administration type: 1=Federal, 2=State, 3=City, 4=Private
tp_localizacao	TINYINT	—	NO	—	School location type: 1=Urban, 2=Rural
in_agua_potavel	TINYINT(1)	1	NO	—	Flag: school has access to drinkable water (0=No, 1=Yes)
in_energia_inexistente	TINYINT(1)	1	NO	—	Flag: school has NO access to electricity (0=No, 1=Yes)
in_esgoto_inexistente	TINYINT(1)	1	NO	—	Flag: school has NO access to sewage system (0=No, 1=Yes)

in_banheiro	TINYINT(1)	1	NO	—	Flag: school has a restroom (0=No, 1=Yes)
in_biblioteca	TINYINT(1)	1	NO	—	Flag: school has a library (0=No, 1=Yes)
in_refeitório	TINYINT(1)	1	NO	—	Flag: school has a canteen (0=No, 1=Yes)
in_computador	TINYINT(1)	1	NO	—	Flag: school has computers (0=No, 1=Yes)
in_internet	TINYINT(1)	1	NO	—	Flag: school has internet access (0=No, 1=Yes)
in_equip_nenhum	TINYINT(1)	1	NO	—	Flag: school has no electronic equipment (0=No, 1=Yes)

Table 5: Table school_census

Variable	Data Type	Length	Nullable	Constraint	Description
school_id	INT	—	NO	PK, AUTO_INCREMENT	Surrogate key generated during ETL
co_municipio	INT	—	NO	FK → municipality	Reference to the city the school belongs to
tp_dependencia	TINYINT	—	NO	—	School administration type: 1=Federal, 2=State, 3=City, 4=Private
tp_localizacao	TINYINT	—	NO	—	School location type: 1=Urban, 2=Rural
in_agua_potavel	TINYINT(1)	1	NO	—	Flag: school has access to drinkable water (0=No, 1=Yes)
in_energia_inexistente	TINYINT(1)	1	NO	—	Flag: school has NO access to electricity (0=No, 1=Yes)

in_esgoto_inexistente	TINYINT(1)	1	NO	—	Flag: school has NO access to sewage system (0=No, 1=Yes)
in_banheiro	TINYINT(1)	1	NO	—	Flag: school has a restroom (0=No, 1=Yes)
in_biblioteca	TINYINT(1)	1	NO	—	Flag: school has a library (0=No, 1=Yes)
in_refeitório	TINYINT(1)	1	NO	—	Flag: school has a canteen (0=No, 1=Yes)
in_computador	TINYINT(1)	1	NO	—	Flag: school has computers (0=No, 1=Yes)
in_internet	TINYINT(1)	1	NO	—	Flag: school has internet access (0=No, 1=Yes)

in Equip Nenhum	TINYINT(1)	1	NO	—	Flag: school has no electronic equipment (0=No, 1=Yes)
------------------------	------------	---	----	---	--

2.4 Relational Data Model

The figure below shows the Entity Relational Diagram (ERD) for dataset School Census.

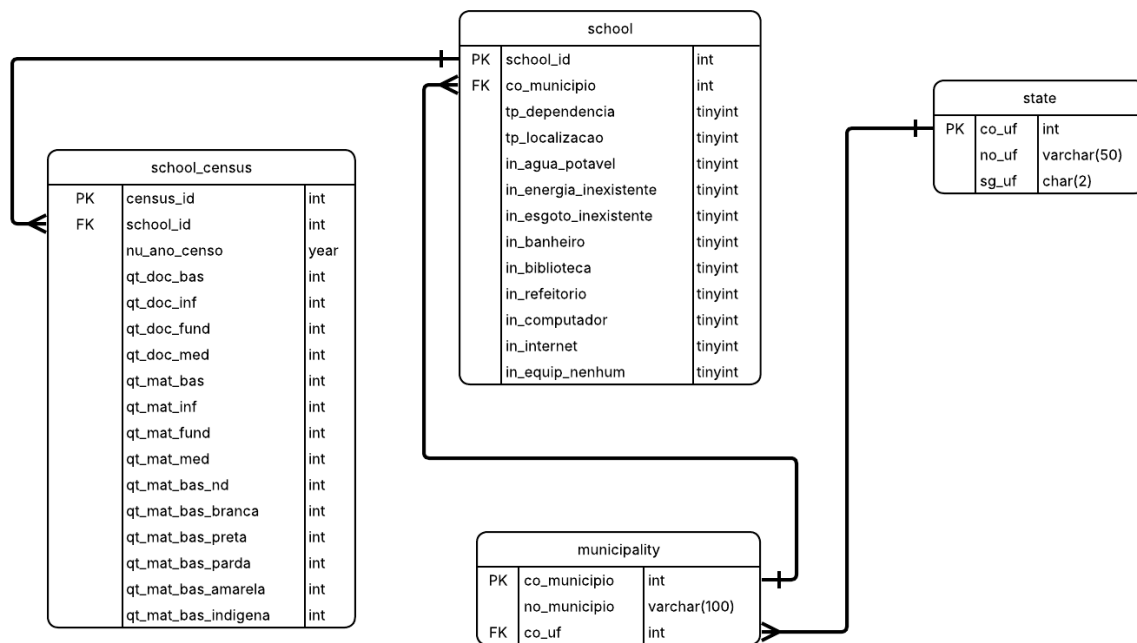


Figure 1 : ERD for School Census

3. Data Preparation

This section explains the key processes during the whole ETL operations. It is implemented as a Python script that orchestrates Apache Spark and PostgreSQL. The pipeline has four sequential stages:

- Extract: Download the yearly CSV files from the Brazilian Open Data Portal and save them locally.
- Convert: Use PySpark to convert each CSV into the columnar Parquet format, dramatically reducing read time.

- Transform: Use PySpark to define fact and dimension tables and join the Parquet data to build a star schema.
- Load: Write each dimension and the fact table into PostgreSQL using Spark's Java Database Connectivity (JDBC) writer and psycopg2 for primary key constraints.

The data is downloaded from INEP website with a total of twelve yearly datasets (2010-2021) where the zip files are extracted and using only the CSV files as the data source using python scripts. The files are then named according to the year the data is based on.

The preprocessing steps uses jupyter notebook as the main interface to run Spark. The CSV files are first loaded into the dataframe object. However, the tutorial pointed out that loading CSV files takes a long time and it was verified to be true during implementation. The tutorial suggests using parquet format before performing any loading. Parquet is a columnar, compressed format that is far better suited to analytical reads. *Figure 2* below is a code snippet of writing loaded CSV dataframe into parquet format.

```
data_csv.write.parquet("./data/censo_escolar.parquet")
```

Figure 2 : Parquet conversion code

The configurations of the dimensions table and fact table were then made, defining its structure and schema where only 31 out of 370 total columns are used for the tutorial. The configuration will be further discussed in the data modelling section. After the configurations were made, it was written to Postgres. *Figure 3* is the code snippet that writes the configurations to Postgres. The code works by extracting unique values of dimension tables' columns, assigning an id to be the primary key and writing the unique values Postgres database. *Figure 3* shows the code snippet to write the dimension table configuration into Postgres.

```

# Write data to Postgres
# Using the configuration in DIMENSION_TABLES_CONFIG
# With id as the primary key

for table_name, table_config in DIMENSION_TABLES_CONFIG.items():

    print(f"[{datetime.now()}] Writing {table_name}")

    data\
    .select(
        [
            F
            .col(field["field"])
            .cast(field["type"])
            .alias(field["field"])

            for field
            in table_config["fields"]
        ]
    )\
    .distinct()\
    .withColumn(
        "id", F.monotonically_increasing_id()
    )\
    .write\
    .jdbc(
        **POSTGRES_CONFIG,
        table=table_name,
        mode="overwrite"
    )

    print(f"[{datetime.now()}] Wrote {table_name}")
    # Define id as the primary key
    cursor = conn.cursor()
    cursor.execute(
        f"ALTER TABLE {table_name} ADD PRIMARY KEY (id);"
    )
    cursor.close()
    conn.commit()

    print(f"[{datetime.now()}] Added primary key to {table_name}")
    print(f"[{datetime.now()}] Done")

```

Figure 3 : Writing dimension tables' configuration to Postgres

The same applies to fact tables that were configured and the schema is written to Postgres adding primary key called *id* to the fact table and adding foreign key constraints on fields

that reference the dimension tables. *Figure 4* shows the code writing fact tables' configuration to Postgres.

```
# Create fact table
# Using the configuration in FACT_CONFIG
# With id as the primary key

# Avoid inserting a backslash into a f-string
comma_break_line = ",\n\t\t\t\t"
facts_table_sql = f"""
CREATE TABLE IF NOT EXISTS {FACT_TABLE_NAME} (
    id SERIAL PRIMARY KEY,
    {
        comma_break_line.join(
            [
                f"{field} INTEGER"
                for field in FACT_COLUMNS
            ]
        )
        + [
            f"ID_{dim_table} BIGINT"
            for dim_table in DIMENSION_ID_CONFIG.keys()
        ]
    }
);

-- Adding Foreign Keys
ALTER TABLE {FACT_TABLE_NAME}
{
    comma_break_line.join(
        [
            f"ADD CONSTRAINT {FACT_TABLE_NAME}_{dim_table}_fk FOREIGN KEY (ID_{dim_table}) REFERENCES {dim_table}(id)"
            for dim_table in DIMENSION_ID_CONFIG.keys()
        ]
    )
}
```

Figure 4 : Writing fact tables' configuration to Postgres

Before the fact table is loaded into Postgres, it needs to be joined facts table dimension IDs and fact column keys extracted in *Figure 5* with dimension tables data from Postgres shown in *Figure 6*.

```
facts_data = data\
    .select(
        [
            *chain(
                *DIMENSION_ID_CONFIG.values(),
                FACT_CONFIG.keys()
            )
        ]
    )
```

Figure 5: Extracting facts table dimension IDs and fact column keys

```

# Joining the id of the dimensions

for table_name, table_fields in DIMENSION_ID_CONFIG.items():

    # Read the dimension data from Postgres
    dim_table = spark.read\
        .jdbc(
            **POSTGRES_CONFIG,
            table=table_name,
        )\
        .withColumnRenamed("id", f"ID_{table_name}")

    # Join the dimension data with the fact data
    facts_data = facts_data\
        .join(
            dim_table,
            on=table_fields,
            how="left"
        )\
        .drop(*table_fields)

```

Figure 6 : Joining data from the facts table dimension IDs and fact column keys with dimension tables data from Postgres

4. Modeling

Only 31 columns are used to generate the star schema shown in *Figure 9*. *Figure 7* shows the columns used to configure the dimension table where the all the columns in the INTEGER_DIMENSIONS list is its own dimension table while DIM_LOCAL table has a few columns that details on geographical locations in Brazil that is present in the dataset. *Figure 8* on the other hand shows the configuration of the fact table where all the columns selected are aggregated data such as total number of teachers and total number of enrollments.

```

INTEGER_DIMENSIONS = [
    "TP_DEPENDENCIA",          # The school administration (state, city, private)
    "TP_LOCALIZACAO",          # The school location (urban rural)
    "IN_AGUA_POTAVEL",         # has access to drinkable water
    "IN_ENERGIA_INEXISTENTE",   # has (NOT) access to energy
    "IN_ESGOTO_INEXISTENTE",    # has (NOT) access to energy
    "IN_BANHEIRO",             # has restroom
    "IN_BIBLIOTECA",           # has library
    "IN_REFEITORIO",           # has canteen
    "IN_COMPUTADOR",           # has computer
    "IN_INTERNET",             # has internet
    "IN_EQUIP_NENHUM"          # no electronic equipment
]

DIMENSION_TABLES_CONFIG = {
    "DIM_LOCAL":{
        "fields": [
            {"field":"NO_UF", "type":"string",},          # State's name
            {"field":"SG_UF", "type":"string",},          # State's abbreviation
            {"field":"CO_UF", "type":"string",},          # State's code
            {"field":"NO_MUNICIPIO", "type":"string",},    # City's name
            {"field":"CO_MUNICIPIO", "type":"string",}     # City's code
        ]
    },
}

DIMENSION_TABLES_CONFIG.update(
{
    "DIM_"+dimension.upper():{
        "fields": [
            {"field":dimension, "type":"integer"}
        ]
    }
    for dimension in INTEGER_DIMENSIONS
}
)

```

Figure 7: Dimension tables' configuration

```

FACT_TABLE_NAME = "FACT_CENSO_ESCOLAR"

FACT_COLUMNS = [
    "QT_DOC_BAS", --# Number of Teachers in the basic education (TOTAL)
    "QT_DOC_INF", --# Number of Teachers in the basic education (child education)
    "QT_DOC_FUND", --# Number of Teachers in the basic education (elementary education)
    "QT_DOC_MED", --# Number of Teachers in the basic education (high school)

    "QT_MAT_BAS", --# Number of enrollments in the basic education (TOTAL)
    "QT_MAT_INF", --# Number of enrollments in the basic education (child education)
    "QT_MAT_FUND", --# Number of enrollments in the basic education (elementary education)
    "QT_MAT_MED", --# Number of enrollments in the basic education (high school)

    "QT_MAT_BAS_ND", --# Number of enrollments in the basic education - Skin color/Race Not Declared
    "QT_MAT_BAS_BRANCA", --# Number of enrollments in the basic education - Skin color/Race Branco
    "QT_MAT_BAS_PRETA", --# Number of enrollments in the basic education - Skin color/Race Preto
    "QT_MAT_BAS_PARDA", --# Number of enrollments in the basic education - Skin color/Race Parada
    "QT_MAT_BAS_AMARELA", --# Number of enrollments in the basic education - Skin color/Race Amarela
    "QT_MAT_BAS_INDIGENA", --# Number of enrollments in the basic education - Skin color/Race Indígena

    "NU_ANO_CENSO" # Census' year
]

FACT_CONFIG = {
    fact:{
        "fields": [
            {"field":fact, "type":"integer"}
        ]
    }
    for fact in FACT_COLUMNS
}

DIMENSION_ID_CONFIG = {
    table_name:[
        field['field']
        for field
        in table_fields['fields']
    ]
    for table_name, table_fields in DIMENSION_TABLES_CONFIG.items()
}

FACT_TABLE_ALL_COLUMNS_ORDERED = FACT_COLUMNS + list(map(lambda col:"ID_"+col, DIMENSION_ID_CONFIG.keys()))

```

Figure 8: Fact tables' configuration

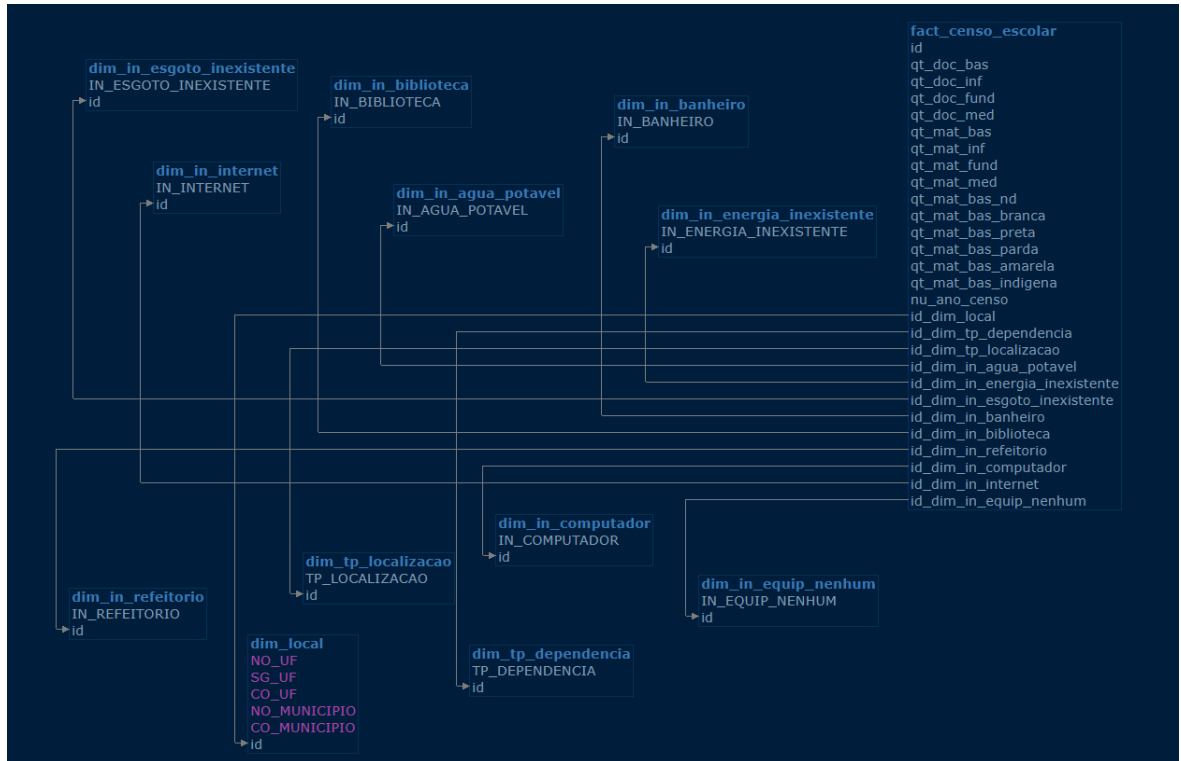


Figure 9: Star schema of school census

5. Evaluation

5.1 Dashboard

To verify that the star schema produces meaningful analytical output, a Metabase dashboard was connected directly to the PostgreSQL database. The dashboard demonstrates business questions can be answered interactively by writing SQL queries in the Metabase.



Figure 10: Dashboard 1 using Metabase

The dashboard above shows total enrollments by year and a card that shows the latest year and its total enrollment.

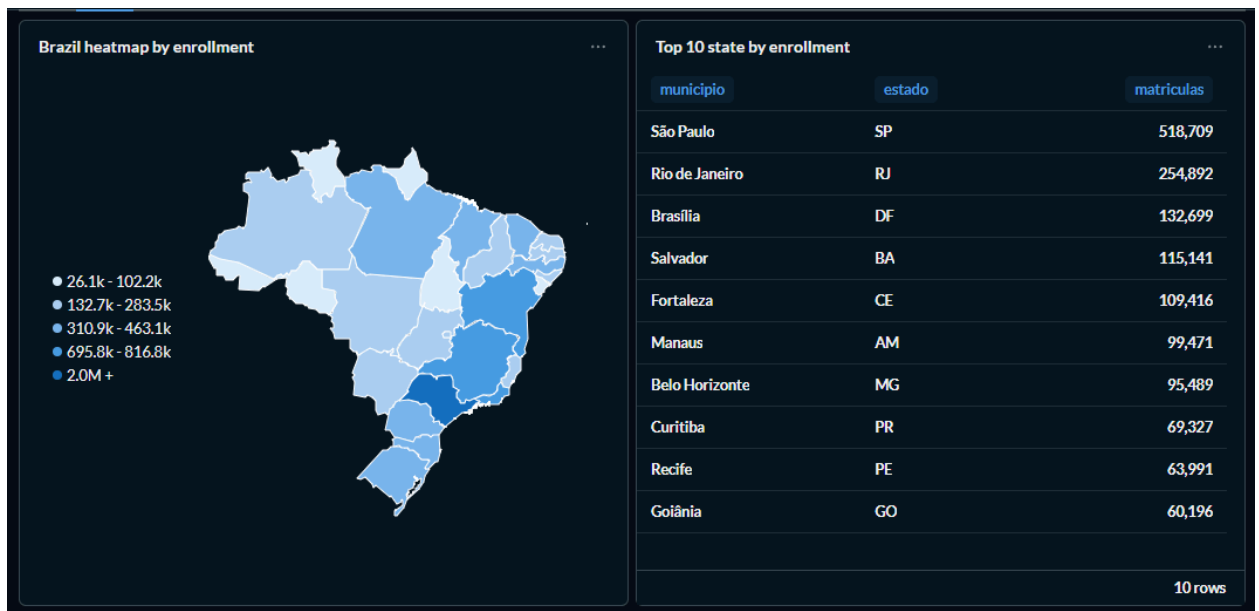


Figure 11: Dashboard 2 using Metabase

The second dashboard above shows the top 10 states by enrollment and the heatmap of the Brazil region that shows each state by their enrollment.

5.2 Limitation

The first limitation in this pipeline is Spark requires a significant amount of driver memory when running locally. The system crashes during the JDBC write stage when loading the fact table that has a massive amount of rows (around 3 million rows) into PostgreSQL. An out-of-memory crash happens at this stage because Spark attempts to collect and push a large partition of data into the JDBC connection all at once. The default partition size is far too large for a machine with limited RAM and increasing `spark.driver.memory` is not always sufficient if the local machine cannot provide it.

The second one is the pipeline has no scheduling mechanism. This means it cannot run automatically on a defined interval. Every time a new yearly census file is released by INEP, a developer must manually trigger the Python script. This makes the pipeline fragile in a production context where if a person forgets to run the code manually, the data become outdated making analysis not very accurate.

5.3 Improvement

The first improvement is to resolve the out-of-memory crash during the JDBC load stage, the pipeline was improved by combining three Spark write options together:

```
facts_data\  
  .select(*FACT_TABLE_ALL_COLUMNS_ORDERED)\  
  .repartition(10)\  
  .write\  
  .option("batchsize", 10000)\  
  .option("numPartitions", 10)\  
  .jdbc(  
    **POSTGRES_CONFIG,  
    table=FACT_TABLE_NAME,  
    mode="append"  
  )
```

Figure 12: Chunking/partition code

`.repartition(10)` splits the DataFrame into 10 balanced partitions before writing to prevent any single partition from becoming too large for driver memory. `.option("numPartitions", 10)` opens exactly 10 parallel JDBC connections to PostgreSQL, one per partition, eliminating connection contention. `.option("batchsize", 10000)` groups rows into batches of 10,000 per INSERT statement to reduce the number of round-trips to the database. The

combination of all three gave more precise control over memory overload during the load phase and successfully eliminated the crash.

The second one is to resolve the scheduling problem by implementing a workflow orchestrator such as Apache Airflow. Airflow would trigger the full pipeline automatically like once per year when INEP publishes the new census release, retry failed tasks and maintain a full execution history so the team can see exactly when each year's data was loaded and see if any stage failed. This transforms the pipeline from a manually-run script into an automatic observable data product.

6. Reflection

Iman Abadi bin Mohd Nizwan

From this tutorial, practical experience in building an ETL pipeline using Apache Spark, from downloading raw CSV data, loading transformed data into PostgreSQL to designing a star schema and visualizing it in Metabase was gained. One of the highlights was understanding Spark's lazy evaluation and how it handles large datasets without crashing memory, which Pandas could not do on this 2.2 GB dataset. A memory issue was also encountered when loading the fact table into PostgreSQL which was resolved using repartition, batchsize and numPartitions options provided deeper understanding of the tutorial by identifying the issue before a solution was applied. Overall, this project provided a practical exposure to tools used in real data engineering work such as Apache Spark, PostgreSQL, Docker and Metabase.

Mohamed Alif Fathi bin Abdul Latif

Through this tutorial, the team gained hands-on experience building a complete ETL pipeline using Apache Spark, from downloading raw CSV data all the way to loading it into a PostgreSQL Star Schema and visualizing it in Metabase. One of the most challenging parts was the installation and configuration process, as setting up PySpark and connecting it to PostgreSQL running inside Docker required careful configuration that was not straightforward for a beginner. Any small misconfiguration, such as a missing JDBC driver or incorrect Docker port mapping, resulted in errors that were difficult to trace. Despite these challenges, completing the setup and seeing the pipeline run successfully end to end was rewarding. This experience taught the team that in data engineering, getting the environment right is just as important as writing the code itself.